

Experiment – 2

Title

Platform Touch Challenge Game in Unity with Color Change

Objective

To design and implement a **3D platform challenge game using Unity**, where players move across platforms and the platform changes color when a player touches it.

Software / Tools Required

- Unity Hub
 - Unity 6.3 LTS
 - Visual Studio / VS Code
 - Windows Operating System
-

Theory

Unity is a powerful **cross-platform game development engine** used to create **2D, 3D, AR, and VR applications**. Unity uses **GameObjects and components** to build interactive scenes.

In this experiment, a **3D platform challenge environment** is created. Multiple player objects move across a series of platforms. When a player lands on a platform, Unity detects the collision and triggers a script that changes the platform color.

This provides **visual feedback** indicating that the platform has been successfully touched.

Unity components used in this experiment include:

Transform

The Transform component controls the **position, rotation, and scale** of objects in the scene.

Rigidbody

The Rigidbody component allows objects to **interact with Unity's physics engine**, including gravity and movement.

Collider

Colliders define the **physical boundaries of objects** and allow collision detection.

C# Scripts

Scripts are used to define **custom behaviors**, such as player movement and platform color changes.

Scene Components Used

The following objects were created in the Unity scene.

Platforms

The platforms act as jumping surfaces for players.

- Platform_1
- Platform_2
- Platform_3
- Platform_4
- Platform_5
- Platform_6
- Platform_7

Each platform contains:

- Transform
 - Mesh Renderer
 - Box Collider
 - PlatformColorChange Script
-

Player Objects

Four player objects are created:

- Capsule_1
- Capsule_2
- Sphere_1
- Cube_Player

Each player object contains:

- Transform
- Collider
- Rigidbody
- PlayerController Script

All player objects are assigned the tag:

Tag → Player

This tag allows platforms to detect collisions specifically with player objects.

Environment Objects

The scene also contains the following objects:

- Main Camera – used to view the scene
- Directional Light – provides illumination
- SafetyGround – prevents players from falling indefinitely
- Canvas – used for UI elements
- EventSystem – handles UI events
- GameManager – manages player references

Steps to Create the Scene

Step 1: Create a New Unity Project

1. Open **Unity Hub**
2. Click **New Project**
3. Select **3D Core Template**
4. Name the project:

ARVR_Lab_Program2

5. Click **Create**
-

Step 2: Create Platforms

1. Right-click in **Hierarchy**
2. Select

3D Object → Cube

3. Rename the object:

Platform_1

4. Adjust the scale in Inspector:

X = 3

Y = 0.5

Z = 3

5. Duplicate the platform using:

Ctrl + D

6. Rename the duplicates:

Platform_2
Platform_3
Platform_4
Platform_5
Platform_6
Platform_7

7. Arrange the platforms in a **zig-zag path** to create a jumping challenge.
-

Step 3: Create Platform Material

1. In **Assets Panel**, right-click
2. Select

Create → Material

3. Rename the material:

PlatformMaterial

4. Change the **Base Map Color** to pink.
 5. Drag the material onto all platforms.
-

Step 4: Create Player Objects

Create the following player objects.

Capsule Player

Hierarchy → Right Click

3D Object → Capsule

Rename:

Capsule_1

Duplicate to create:

Capsule_2

Sphere Player

Hierarchy → Right Click

3D Object → Sphere

Rename:

Sphere_1

Cube Player

Hierarchy → Right Click

3D Object → Cube

Rename:

Cube_Player

Step 5: Add Rigidbody to Players

Select each player object and add Rigidbody.

Inspector → Add Component → Rigidbody

Settings:

Mass = 1
Use Gravity = ON
Is Kinematic = OFF

Step 6: Assign Player Tag

Select each player object and assign the tag.

Tag → Player

If the tag does not exist:

Tag → Add Tag → Player

Then assign it to all player objects.

Step 7: Create Safety Ground

1. Right-click in Hierarchy
2. Select

3D Object → Plane

Rename:

SafetyGround

Set scale:

10 , 1 , 10

This prevents players from falling infinitely.

Scripts Used

Three scripts are used in this experiment.

Script 1 – PlayerController.cs

Purpose

This script controls **player movement**.

Players move using keyboard inputs and interact with platforms using Rigidbody physics.

Script Attached To

This script is attached to:

- Capsule_1
 - Capsule_2
 - Sphere_1
 - Cube_Player
-

Script Code

- using UnityEngine;
-
- public class PlayerController : MonoBehaviour
- {
- public float moveSpeed = 6f;
- private Rigidbody rb;
- private Vector3 startPosition;
-
- void Start()
- {

- rb = GetComponent<Rigidbody>();
- startPosition = transform.position;
- }
-
- void FixedUpdate()
- {
- float moveX = Input.GetAxis("Horizontal");
- float moveZ = Input.GetAxis("Vertical");
-
- float moveY = 0;
-
- if (Input.GetKey(KeyCode.E))
- moveY = 1;
-
- if (Input.GetKey(KeyCode.Q))
- moveY = -1;
-
- Vector3 movement = new Vector3(moveX, moveY, moveZ);
-
- rb.MovePosition(rb.position + movement * moveSpeed * Time.fixedDeltaTime);
- }
-
- public void ResetPosition()
- {
- rb.position = startPosition;
- rb.linearVelocity = Vector3.zero;
- }
- }

Script 2 – PlatformColorChange.cs

Purpose

This script changes the **platform color** when a **player touches the platform**.

Script Attached To

This script is attached to all platforms:

- Platform_1
 - Platform_2
 - Platform_3
 - Platform_4
 - Platform_5
 - Platform_6
 - Platform_7
-

Script Code

```
• using UnityEngine;
•
• public class PlatformColorChange : MonoBehaviour
• {
•     private Renderer rend;
•
•     void Start()
•     {
•         rend = GetComponent<Renderer>();
•     }
•
•     void OnCollisionEnter(Collision collision)
•     {
•         if (collision.gameObject.CompareTag("Player"))
•         {
•             rend.material.color = Color.green;
•         }
•     }
• }
```

GameManager

The GameManager object manages references to all players.

Players included:

Capsule_1
Capsule_2
Sphere_1
Cube_Player

It can be used to manage game logic or reset players.

Working

1. The Unity scene starts with player objects placed near the first platform.
 2. Players move using keyboard controls.
 3. When a player lands on a platform, Unity detects a collision.
 4. The PlatformColorChange script is triggered.
 5. The platform color changes from **pink to green**.
 6. This indicates that the player successfully touched the platform.
-

Output

- A 3D platform challenge environment is created.
- Multiple player objects move across platforms.
- When a player lands on a platform:

Platform color changes to green.

- Collision detection works correctly.
-

Result

The **Platform Touch Challenge Game** was successfully implemented using Unity. Platforms correctly detect player collisions and dynamically change color using C# scripts.

Quick Recap to Recreate Experiment

1. Create a **new 3D Unity project**.
2. Create **7 cube platforms** and arrange them in a zig-zag path.
3. Create **players**: Capsule_1, Capsule_2, Sphere_1, Cube_Player.
4. Add **Rigidbody** to all player objects.
5. Assign **Tag = Player** to all players.
6. Create a **pink material** and apply it to all platforms.
7. Create **SafetyGround (Plane)** below platforms.
8. Create and attach scripts:
 - **PlayerController.cs** → **attach to all players** (movement)
 - **PlatformColorChange.cs** → **attach to all platforms** (change color on collision)
 - **PlatformTouch.cs** → **optional, prints console message**
9. Press **Play** and move players across platforms.
10. When player touches a platform → **platform color changes to green**.

Viva Questions

1. What is Unity?

Answer:

Unity is a **cross-platform game development engine** used to develop **2D, 3D, Augmented Reality (AR), and Virtual Reality (VR) applications** using C# scripting.

2. What is a GameObject in Unity?

Answer:

A GameObject is the **basic object in Unity scenes** that represents characters, lights, cameras, platforms, or any object in the environment.

3. What is a Component?

Answer:

Components are features that define the **behavior and properties of a GameObject**.

Examples include:

- Transform
 - Rigidbody
 - Collider
 - Scripts
-

4. What is a Rigidbody?

Answer:

Rigidbody is a Unity component that allows objects to **interact with the physics system**, including gravity, movement, and collisions.

5. What is a Collider?

Answer:

A Collider is a component used to **detect collisions between objects** in Unity.

Common types include:

- Box Collider
 - Sphere Collider
 - Capsule Collider
-

6. What is **OnCollisionEnter()**?

Answer:

OnCollisionEnter() is a Unity function that is automatically called when **two objects with colliders collide**.

Example:

```
void OnCollisionEnter(Collision collision)
```

This function is used to detect interactions between objects.

7. Why is Tag used in Unity?

Answer:

Tags are used to **identify specific types of objects** in a scene.

In this experiment, the tag **Player** is assigned to player objects so that platforms can detect when a player touches them.

8. Why is **CompareTag()** used?

Answer:

CompareTag() is used to check whether a collided object has a specific tag.

Example:

```
collision.gameObject.CompareTag("Player")
```

This ensures the platform reacts **only when a player touches it**.

9. Why does the platform change color?

Answer:

The platform changes color to **visually indicate that the player has successfully landed on the platform**.

10. Why is **FixedUpdate()** used in the PlayerController script?

Answer:

FixedUpdate() is used for **physics-based operations** such as Rigidbody movement because it runs at **fixed time intervals**, making physics calculations more stable.

11. What is the purpose of the SafetyGround?

Answer:

The SafetyGround prevents players from **falling infinitely if they miss a platform**.

12. What is the difference between **Update()** and **FixedUpdate()**?

Answer:

Update()

FixedUpdate()

Called every frame

Called at fixed intervals

Used for input and UI

Used for physics operations

Frame dependent

Physics dependent